

University Courses Scheduling Tool

Software Requirements Specification

Date:

August 14, 2025

Name:

Yasir Alobaid

Prepared for

University Internship Program at QUIT

Instructor: Abdullatif Alabdullatif

Revision History

Date	Description	Author	Comments
Aug 14, 2025	Version 1	Yasir Alobaid	Initial Release

Table of Contents

REVISION HISTORY	II
.1 INTRODUCTION	1
1.1 PURPOSE	1
1.2 SCOPE.....	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	1
1.4 REFERENCES	2
1.5 OVERVIEW	2
.....	2
2. GENERAL DESCRIPTION	3
2.1 PRODUCT PERSPECTIVE & DEVELOPMENT TOOLS	3
2.2 PRODUCT FUNCTIONS	3
2.3 USER CHARACTERISTICS.....	3
2.4 DESIGN AND IMPLEMENTATION CONSTRAINTS	4
2.5 ASSUMPTIONS AND DEPENDENCIES	4
3. SPECIFIC REQUIREMENTS.....	4
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	4
3.1.1 <i>User Interfaces</i>	4
3.1.2 <i>Software Interfaces</i>	4
3.1.3 <i>Communications Interfaces</i>	4
3.2 FUNCTIONAL REQUIREMENTS.....	4
3.3 USE CASES.....	5
3.3.1 <i>Use Case UC-01</i>	5
3.3.2 <i>Use Case UC-02</i>	5
3.3.3 <i>Use Case UC-03</i>	5
3.3.4 <i>Use Case UC-04</i>	5
3.3.5 <i>Use Case UC-05</i>	5
3.5 NON-FUNCTIONAL REQUIREMENTS	5
3.5.1 <i>Performance</i>	5
3.5.2 <i>Security</i>	5
3.5.3 <i>Usability</i>	5
3.5.4 <i>Portability</i>	5
3.6 OTHER REQUIREMENTS.....	5
4. ANALYSIS MODELS	6
.....	6
.....	6
4.2 ENTITY-RELATIONSHIP DIAGRAM (ERD).....	7
5. CHANGE MANAGEMENT PROCESS	7
A. APPENDICES	8
A.1 APPENDIX 1 SCHEDULE.BLADE.PHP (MAIN PAGE).....	9
A.2 APPENDIX 2 COURSESEEDER.PHP (DATABASE USED).....	17

1. Introduction

1.1 Purpose

This document provides a comprehensive description of the *University Timetable Management Tool*.

The tool is designed to allow students to easily view, select, and manage their available courses and sections for the semester, using a digital timetable interface. The primary aim is to improve the efficiency of course registration, reduce scheduling conflicts, and provide a printable final schedule.

1.2 Scope

*Currently, the system uses **SQLite** as a local database to store course and user data. In the future, it will be upgraded to connect directly to the **University's API** once access permissions are granted, allowing real-time fetching of a student's available courses from the university system.*

Main features include:

- Student login (with predefined credentials in SQLite for now).
- Display of available courses and sections linked to the logged-in student.
- Interactive timetable interface to arrange chosen sections.
- Ability to remove or change selections by clicking on timetable entries.
- Printing the final schedule (including course and section numbers) for submission to the official university system.

1.3 Definitions, Acronyms, and Abbreviations

- **Students:** To arrange their schedules quickly.
- **University Administration:** To evaluate the project as a candidate for official adoption.
- **System Developers:** To maintain, improve, and integrate the tool with the university's backend.
- **Login** → Student enters credentials.
- **Course List** → Left side shows available courses.
- **Select Section** → Clicking a section places it in the timetable grid.
- **Modify Schedule** → Clicking an entry in the timetable removes it.
- **Print Schedule** → Produces a ready-to-print page containing all selected courses & section numbers.

1.4 References

- IEEE 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Laravel Framework Documentation.
- SQLite Documentation.

1.5 Overview

After logging in with their student credentials, the user is redirected to the main timetable interface.

The left panel displays **only the courses available to that student**, fetched from SQLite (temporary) or the API (future).

Below each course, available **sections** are listed.

2. General Description

2.1 Product Perspective & Development Tools

The University Timetable Management Tool is a web-based application designed to integrate into the university's existing portal. At present, it operates as a standalone module using a local SQLite database. In the future, it will connect to the university's course API for real-time course and section retrieval.

Technologies & Frameworks

Component Technology

Backend PHP 8, Laravel Framework

Frontend HTML, CSS, JavaScript (AJAX, jQuery)

Database SQLite (with possible migration to MySQL)

Development Laravel Artisan CLI, Composer

IDE Visual Studio Code

Version Control GitHub

Testing Tools Chrome/Firefox Developer Tools

2.2 Product Functions

- Authenticate students
- Fetch and display offered courses & sections
- Manage schedule by adding/removing sections
- Export schedule as printable image

2.3 User Characteristics

- Students familiar with web interfaces
- Basic computer literacy
- Internet access required

2.4 Design and Implementation Constraints

- Limited to university access
- SQLite database storage not APIs
- Browser-based only

2.5 Assumptions and Dependencies

- Assumes correct credentials are provided by university
- Dependent on API availability for fetching course data

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Page	Elements	Description
Login Page	Username, Password fields	Authenticates students
Schedule Page	Table of courses, Add/Remove buttons, Print button	Allows schedule editing
Print View	Read-only table with course & section IDs	For submission/printing

3.1.2 Software Interfaces

Laravel backend, SQLite connection via Laravel's Eloquent ORM.

3.1.3 Communications Interfaces

Localhost network for development.
HTTP for deployment.

3.2 Functional Requirements

FR-01 The system shall allow students to log in using their university credentials.

FR-02 The system shall retrieve all courses available to the logged-in student.

FR-03 The system shall allow adding a course section to the schedule.

FR-04 The system shall allow removing a course section from the schedule.

FR-05 The system shall generate a printable schedule in image or PDF format.

3.3 Use Cases

3.3.1 Use Case UC-01: Student logs in

3.3.2 Use Case UC-02: Student views available courses

3.3.3 Use Case UC-03: Student adds a course to schedule

3.3.4 Use Case UC-04: Student removes a course from schedule

3.3.5 Use Case UC-05: Student prints schedule

3.4 Non-Functional Requirements

3.4.1 Performance

- The system shall load any page in under 2 seconds on standard hardware.

3.4.2 Security

- Passwords stored hashed in the database.
- Session-based authentication.

3.4.3 Usability

- Interface shall be simple and intuitive.
- Compatible with modern browsers.

3.4.4 Portability

- Works on all modern browsers

3.5 Other Requirements

- Future integration with university APIs when access becomes available.
- Role-based access control for admin features.

4. Analysis Models

This section provides the analysis models used in developing the functional requirements described in Section 3.

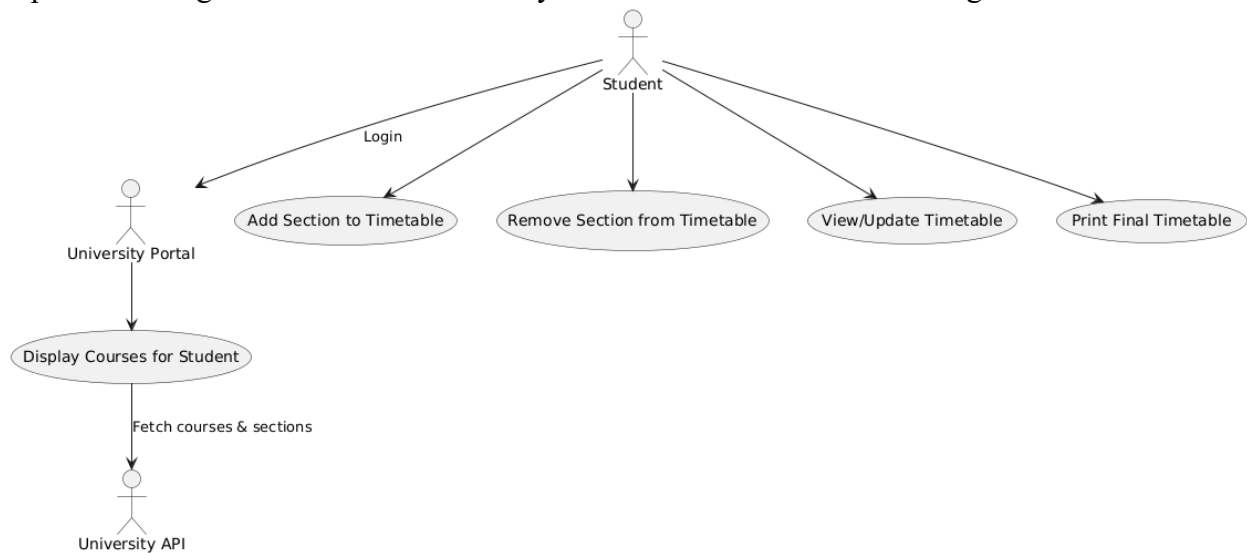
Each model is traceable to the system requirements and helps visualize the structure and behavior of the University Timetable Assistant.

4.1 Use Case Diagram

Introduction:

The Use Case Diagram illustrates the main interactions between the Student, University Portal, University API, and the Timetable Assistant tool.

It provides a high-level view of how the system is used to achieve its main goals.

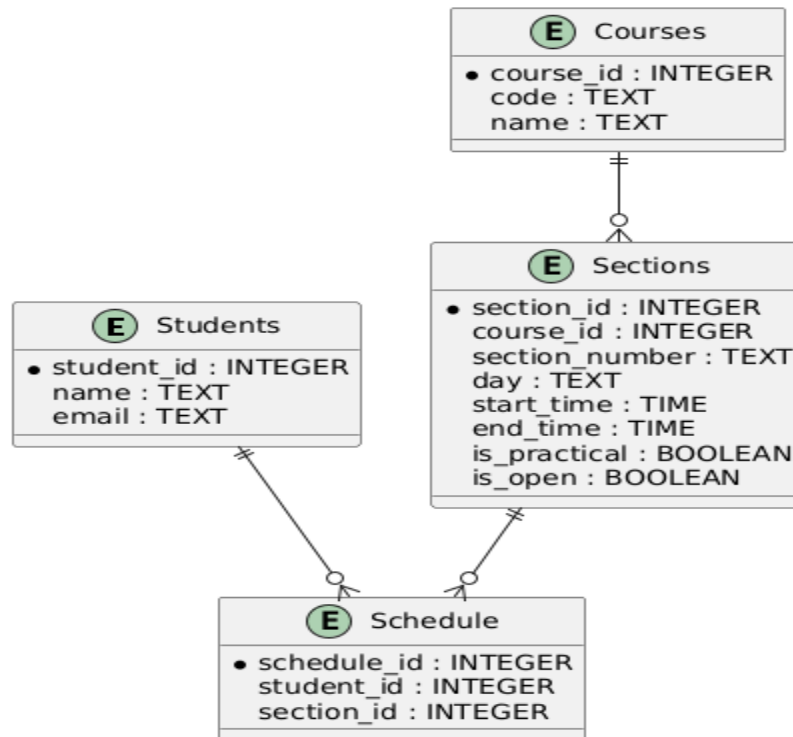


4.2 Entity-Relationship Diagram (ERD)

Introduction:

The ERD describes the data structure of the Timetable Assistant, including the relationships between Students, Courses, Sections, and Schedules.

It shows how student-specific course data is linked and stored.



5. Change Management Process

Introduction:

The Change Management Process ensures that any changes to the system's requirements, design, or implementation are properly reviewed, approved, and documented.

Process:

1. **Request for Change (RFC):** A formal request is submitted by stakeholders (student services, IT department, or project owner).
2. **Impact Analysis:** The development team evaluates the impact of the change on existing requirements, schedule, and resources.
3. **Approval:** The change is reviewed by the University IT Manager or project sponsor.
4. **Implementation:** Approved changes are implemented in the development environment.
5. **Testing:** QA team tests the new changes to ensure they meet requirements without breaking existing functionality.
6. **Deployment:** Changes are deployed to production after final approval.
7. **Documentation Update:** SRS and technical documentation are updated to reflect the change.

A. Appendices

File	Function Name	Description
<code>login.php</code>	<code>loginStudent(\$username, \$password)</code>	Validates student credentials and starts session.
<code>CourseController.php</code>	<code>getCoursesByStudent(\$student_id)</code>	Retrieves available courses for a student.
<code>CourseController.php</code>	<code>getSectionsByCourse(\$course_id)</code>	Retrieves sections for a given course.
<code>ScheduleController.php</code>	<code>addToSchedule(\$student_id, \$section_id)</code>	Adds a selected section to the student's timetable.
<code>ScheduleController.php</code>	<code>removeFromSchedule(\$student_id, \$section_id)</code>	Removes a section from the timetable.
<code>print.php</code>	<code>generatePrintableSchedule(\$student_id)</code>	Generates a printable view of the student's schedule.

A.1 Appendix 1 schedule.blade.php (main page)

```
@extends('layouts.app')

@section('content')
<div class="min-h-screen bg-gradient-to-br from-blue-50 to-white py-10">
    <div class="max-w-6xl mx-auto">
        <div class="flex justify-end mb-4">
            <form method="POST" action="{{ route('logout') }}">
                @csrf
                <button type="submit" class="px-5 py-2 bg-red-600 text-white
rounded-xl shadow hover:bg-red-800 transition-all duration-150">العودة الى
myqu</button>
            </form>
        </div>
        <h1 class="text-3xl font-extrabold mb-8 text-gray-800 text-center
tracking-tight drop-shadow">جدول التسجيل</h1>
        <div class="flex flex-col md:flex-row gap-8">
            <!-- قائمة المقررات -->
            <div class="md:w-1/4 w-full bg-white border rounded-2xl shadow-lg p-5
mb-6 md:mb-0">
                <h2 class="text-gray-800 text-2xl font-bold mb-6 text-
center">المقررات</h2>
                @foreach($courses as $course)
                    <div class="mb-4">
                        <button onclick="toggleSections('course-{{ $course->id
}}')" class="w-full font-bold bg-gray-100 border border-gray-300 p-3 rounded-xl
text-gray-800 shadow-sm text-center text-lg hover:bg-gray-200 transition-colors
duration-150">
                            ({{ $course->code }})<br>
                            <span class="text-base">{{ $course->name }}</span>
                        </button>
                        <div id="course-{{ $course->id }}" class="ml-2 mt-2
hidden">
                            @foreach($course->sections as $section)
                                <button class="section-btn mt-2 px-3 py-2
rounded-xl border text-sm w-full text-right transition-all duration-150 {{
$section->is_closed ? 'border-red-200 bg-red-50 cursor-not-allowed opacity-75' :
'border-blue-200 bg-gray-50 hover:bg-blue-100 hover:scale-105' }} shadow-sm"
                                    data-section='@json($section)'
                                    data-course='@json($course)'
                                    {{ $section->is_closed ? 'disabled' : '' }}>
```

```

    {{ $section->id }} - {{ __($section->day)
  }}<br>
    <span class="text-xs">{{ $section->start_time
  }}-{{ $section->end_time }}</span>
    @if($section->is_closed)<span class="text-
red-500 text-xs block">مغلقة</span>@endif
  </button>
  @endforeach
</div>
</div>
  @endforeach
</div>
<!-- جدول الأسبوع -->
<div class="md:w-3/4 w-full">
  <div class="flex flex-row justify-between items-center mb-4">
    <h2 class="text-gray-800 text-2xl font-bold text-center">جدول
الأسبوع</h2>
    <button id="download-image" class="px-5 py-2 bg-gray-800
text-white rounded-xl shadow hover:bg-black transition-all duration-150">طباعة
صورة للجدول</button>
  </div>
  <div class="overflow-x-auto rounded-2xl shadow-lg bg-white">
    <table id="schedule-table-image" class="w-full border text-
center rounded-2xl overflow-hidden bg-white">
      <thead>
        <tr class="bg-blue-50">
          <th class="border p-3 text-lg">الوقت</th>
          <th class="border p-3 text-lg">الأحد</th>
          <th class="border p-3 text-lg">الاثنين</th>
          <th class="border p-3 text-lg">الثلاثاء</th>
          <th class="border p-3 text-lg">الأربعاء</th>
          <th class="border p-3 text-lg">الخميس</th>
        </tr>
      </thead>
      <tbody id="schedule-table">
        @php
          $days =
[ 'Sunday'=>'الأحد', 'Monday'=>'الاثنين', 'Tuesday'=>'الثلاثاء', 'Wednesday'=>'الأربعاء', 'Thursda
y'=>'الخميس' ];

          $fixedTimeSlots = [
            ['start' => '08:00:00', 'end' => '10:00:00'],
            ['start' => '09:00:00', 'end' => '10:30:00'],
            ['start' => '11:00:00', 'end' => '12:30:00'],
            ['start' => '12:00:00', 'end' => '14:00:00'],
            ['start' => '13:00:00', 'end' => '14:30:00'],

```

[illegible]

```

window.schedule = {};

// Load registered sections from backend
const registeredSections = @json($registeredSections ?? []);

document.getElementById('download-image').onclick = function() {
  const table = document.getElementById('schedule-table-image');
  html2canvas(table).then(function(canvas) {
    const link = document.createElement('a');
    link.download = 'schedule.png';
    link.href = canvas.toDataURL();
    link.click();
  });
};

// --- Interactive scheduling logic ---
const schedule = {}; // { day: { time: {course, section} } }
const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday'];
const sectionButtons = document.querySelectorAll('.section-btn');
const table = document.getElementById('schedule-table');
const csrfToken = document.querySelector('meta[name="csrf-token"]').getAttribute('content');

// Helper: Check for time conflict
function hasConflict(day, start, end) {
  if (!schedule[day]) schedule[day] = {};

  // Convert times to minutes for easier comparison
  start = start.split(':').map(Number);
  end = end.split(':').map(Number);
  const startTime = start[0] * 60 + start[1];
  const endTime = end[0] * 60 + end[1];

  for (const slot in schedule[day]) {
    const [slotStart, slotEnd] = slot.split('-');
    const [sHour, sMin] = slotStart.split(':').map(Number);
    const [eHour, eMin] = slotEnd.split(':').map(Number);
    const slotStartTime = sHour * 60 + sMin;
    const slotEndTime = eHour * 60 + eMin;

    // Check if times overlap
    if (!(endTime <= slotStartTime || startTime >= slotEndTime)) {
      return true;
    }
  }
}

```

```

    return false;
}

// Helper: Add section to schedule (with optimistic update)
function addSectionToSchedule(section, course) {
    const day = section.day;
    const start = section.start_time;
    const end = section.end_time;
    const slotKey = `${start}-${end}`;
    if (!schedule[day]) schedule[day] = {};

    // Optimistically update UI first
    schedule[day][slotKey] = {course, section};
    const cell = table.querySelector(`tr[data-time='${slotKey}'] td[data-day='${day}']`);
    if (cell) {
        cell.innerHTML = `<div class='bg-blue-200 rounded p-1 cursor-pointer remove-course' data-day='${day}' data-slot='${slotKey}'>${course.code}<br><span class='text-xs'>${section.id}</span></div>`;
    }

    // Then sync with backend
    fetch('/schedule/register', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': csrfToken
        },
        body: JSON.stringify({ section_id: section.id })
    })
    .then(res => {
        if (!res.ok) {
            // If error, rollback the UI change
            delete schedule[day][slotKey];
            if (cell) cell.innerHTML = '';
            return res.json().then(data => { throw data; });
        }
        return res.json();
    })
    .catch(err => {
        alert(err.message || 'حدث خطأ أثناء إضافة المقرر');
    });
}

// Helper: Remove section from schedule (with optimistic update)

```

```

async function removeSectionFromSchedule(day, slotKey) {
  const section = schedule[day][slotKey]?.section;
  if (!section) return;

  // Store current state for potential rollback
  const oldState = schedule[day][slotKey];
  const cell = table.querySelector(`tr[data-time='${slotKey}'] td[data-day='${day}']`);
  const oldHtml = cell?.innerHTML;

  // Optimistically update UI first
  delete schedule[day][slotKey];
  if (cell) cell.innerHTML = '';

  try {
    const res = await fetch('/schedule/remove', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'X-CSRF-TOKEN': csrfToken
      },
      body: JSON.stringify({ section_id: section.id })
    });

    if (!res.ok) {
      // If error, rollback the UI change
      schedule[day][slotKey] = oldState;
      if (cell) cell.innerHTML = oldHtml;
      throw new Error('فشل في حذف المقرر');
    }
  } catch (err) {
    // Rollback on network error
    schedule[day][slotKey] = oldState;
    if (cell) cell.innerHTML = oldHtml;
    alert(err.message || 'حدث خطأ أثناء حذف المقرر');
  }
}

// Load and display registered sections
if (registeredSections && Array.isArray(registeredSections)) {
  registeredSections.forEach(section => {
    if (!section || !section.course) return;

    const day = section.day;
    const start = section.start_time;

```

```

    const end = section.end_time;
    const slotKey = `${start}-${end}`;

    // Initialize the schedule object properly
    if (!schedule[day]) schedule[day] = {};
    schedule[day][slotKey] = {
        course: section.course,
        section: section
    };

    // Update the UI
    const cell = table.querySelector(`tr[data-time='${slotKey}'] td[data-day='${day}']`);
    if (cell) {
        cell.innerHTML = `<div class='bg-blue-200 rounded p-1 cursor-pointer remove-course' data-day='${day}' data-slot='${slotKey}'>${section.course.code}<br><span class='text-xs'>${section.id}</span></div>`;
    }
    });
}

// Section button click
sectionButtons.forEach(btn => {
    btn.addEventListener('click', function() {
        const section = JSON.parse(this.getAttribute('data-section'));
        const course = JSON.parse(this.getAttribute('data-course'));

        // Prevent adding closed sections
        if (section.is_closed) {
            alert('هذا القسم مغلق ولا يمكن التسجيل فيه');
            return;
        }

        const day = section.day;
        const start = section.start_time;
        const end = section.end_time;
        const slotKey = `${start}-${end}`;

        // Prevent adding if already scheduled
        if (schedule[day] && schedule[day][slotKey]) {
            alert('تمت إضافة هذا القسم بالفعل في هذا الوقت');
            return;
        }
        // Prevent time conflict

```

```
        if (hasConflict(day, start, end)) {
            alert('يوجد تعارض في الوقت مع مقرر آخر');
            return;
        }
        addSectionToSchedule(section, course);
    });
});

// Remove course from schedule on cell click
table.addEventListener('click', function(e) {
    if (e.target.classList.contains('remove-course')) {
        const day = e.target.getAttribute('data-day');
        const slotKey = e.target.getAttribute('data-slot');
        removeSectionFromSchedule(day, slotKey);
    }
});
});
</script>
@endsection
```

A.2 Appendix 2 CourseSeeder.php (database used)

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Course;
use App\Models\Section;

class CourseSeeder extends Seeder
{
    public function run()
    {
        // Ahmed's Computer Science Courses
        // CS101 - Introduction to Programming
        $course = Course::create(['code' => 'CS101', 'name' => 'مقدمة في علوم
الحاسب']);
        $course->sections()->create(['id' => 1001, 'day' => 'Sunday',
'start_time' => '08:00:00', 'end_time' => '09:30:00']);
        $course->sections()->create(['id' => 1002, 'day' => 'Monday',
'start_time' => '11:00:00', 'end_time' => '12:30:00', 'is_closed' => true]);

        // CS302 - مدخل إلى هندسة البرمجيات
        $course = Course::create(['code' => 'CS302', 'name' => 'مدخل إلى هندسة
البرمجيات']);
        $course->sections()->create(['id' => 1101, 'day' => 'Monday',
'start_time' => '09:00:00', 'end_time' => '10:30:00']);
        $course->sections()->create(['id' => 1102, 'day' => 'Wednesday',
'start_time' => '13:00:00', 'end_time' => '14:30:00']);

        // COE401 - التصميم المنطقي
        $course = Course::create(['code' => 'COE401', 'name' => 'التصميم المنطقي']);
        $course->sections()->create(['id' => 1201, 'day' => 'Tuesday',
'start_time' => '08:00:00', 'end_time' => '09:30:00']);
        $course->sections()->create(['id' => 1202, 'day' => 'Thursday',
'start_time' => '11:00:00', 'end_time' => '12:30:00', 'is_closed' => true]);

        // CS204 - هياكل البيانات
        $course = Course::create(['code' => 'CS204', 'name' => 'هياكل البيانات']);
        $course->sections()->create(['id' => 1301, 'day' => 'Sunday',
'start_time' => '11:00:00', 'end_time' => '12:30:00']);
        $course->sections()->create(['id' => 1302, 'day' => 'Tuesday',
'start_time' => '13:00:00', 'end_time' => '14:30:00']);
    }
}
```

```

// CS303 - خوارزميات
$course = Course::create(['code' => 'CS303', 'name' => 'خوارزميات']);
$course->sections()->create(['id' => 1401, 'day' => 'Monday',
'start_time' => '13:00:00', 'end_time' => '14:30:00']);
$course->sections()->create(['id' => 1402, 'day' => 'Wednesday',
'start_time' => '09:00:00', 'end_time' => '10:30:00', 'is_closed' => true]);

// CS304 - أنظمة التشغيل
$course = Course::create(['code' => 'CS304', 'name' => 'أنظمة التشغيل']);
$course->sections()->create(['id' => 1501, 'day' => 'Thursday',
'start_time' => '08:00:00', 'end_time' => '09:30:00']);
$course->sections()->create(['id' => 1502, 'day' => 'Tuesday',
'start_time' => '11:00:00', 'end_time' => '12:30:00']);

// Saeed's Medical Courses
// MED101 - مقدمة في الطب
$course = Course::create(['code' => 'MED101', 'name' => 'مقدمة في الطب']);
$course->sections()->create(['id' => 2001, 'day' => 'Sunday',
'start_time' => '08:00:00', 'end_time' => '09:30:00']);
$course->sections()->create(['id' => 2002, 'day' => 'Tuesday',
'start_time' => '11:00:00', 'end_time' => '12:30:00']);

// ANAT201 - علم التشريح البشري
$course = Course::create(['code' => 'ANAT201', 'name' => 'علم التشريح
البشري']);
$course->sections()->create(['id' => 2101, 'day' => 'Monday',
'start_time' => '09:00:00', 'end_time' => '10:30:00']);
$course->sections()->create(['id' => 2102, 'day' => 'Wednesday',
'start_time' => '13:00:00', 'end_time' => '14:30:00', 'is_closed' => true]);

// MED301 - الطب الباطني
$course = Course::create(['code' => 'MED301', 'name' => 'الطب الباطني']);
$course->sections()->create(['id' => 2201, 'day' => 'Tuesday',
'start_time' => '08:00:00', 'end_time' => '09:30:00']);
$course->sections()->create(['id' => 2202, 'day' => 'Thursday',
'start_time' => '11:00:00', 'end_time' => '12:30:00']);

// PHYSI202 - علم الفسيولوجيا
$course = Course::create(['code' => 'PHYSI202', 'name' => 'علم الفسيولوجيا']);
$course->sections()->create(['id' => 2301, 'day' => 'Sunday',
'start_time' => '11:00:00', 'end_time' => '12:30:00']);
$course->sections()->create(['id' => 2302, 'day' => 'Wednesday',
'start_time' => '09:00:00', 'end_time' => '10:30:00']);
}
}

```